DivAi

Architectural & System Design Document

Website Intelligence → Use Cases → Business Solutions → Project Proposals

Document Version	v1.0 — Initial Release
Status	Draft — For Internal Review
Prepared For	DivAi Engineering Team
Stack	Python · LangChain · LangGraph · RAG · TypeScript · FastAPI
Date	10 March 2026

1. Executive Summary

DivAi is an AI-powered business intelligence and solution scoping platform built for a custom software development agency. It enables both internal teams and client-facing workflows to: scrape a target website's DOM structure and network traffic, extract meaningful signals, identify high-value business use cases, recommend tailored software solutions, and auto-generate technical project proposal documents — all driven by a multi-agent AI pipeline.

> Core Value Proposition:
> Turn any website into a qualified business development lead — with a ready-to-build technical proposal — in under 5 minutes.

This document covers the complete system architecture, agent design, data flows, API contracts, infrastructure decisions, and a phased delivery roadmap based on the learning stack defined in the AI Development Guide (Python, LangChain, LangGraph, RAG, TypeScript).

2. System Overview

2.1 High-Level Architecture

DivAi is structured as four functional layers that work sequentially, with a multi-agent orchestration core:

Layer	Responsibility	Primary Technology
1. Ingestion Layer	Scrape website DOM, intercept network requests, extract raw signals	Playwright, Python, Puppeteer
2. Intelligence Layer	Analyse signals, classify patterns, generate use cases with confidence scores	LangGraph Agents, Claude API
3. Solution Layer	Match use cases to solution templates, score effort/value, select builds	LangChain, RAG (ChromaDB)
4. Proposal Layer	Generate structured technical proposal documents in multiple formats	LangGraph, python-docx, Notion API

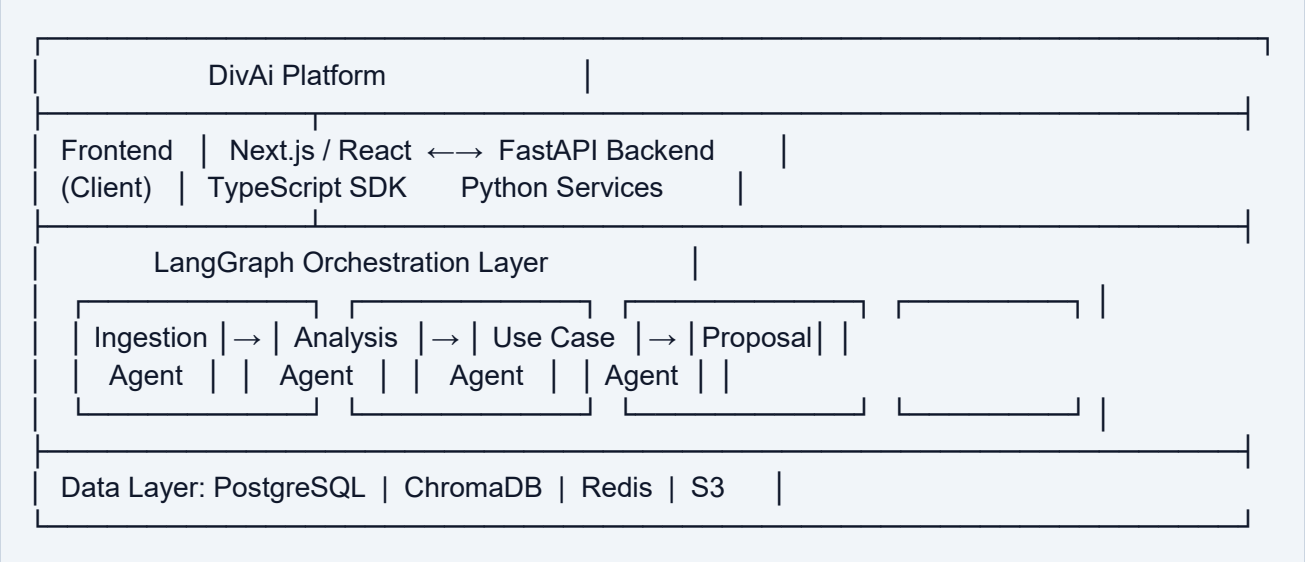
2.2 User Journey

There are two primary user personas: Internal (dev/BD team) and Client-Facing (client self-serve or guided demo). The workflow is identical for both, with role-based access controlling which steps are editable.

1. User submits a target URL via the DivAi web interface
2. Ingestion Agent scrapes the website (DOM + network layer)

3. Analysis Agent processes raw data and extracts structured signals
4. Use Case Agent generates ranked, confidence-scored business opportunities
5. User reviews and selects use cases they want to pursue
6. Solution Agent matches selected use cases to software solutions from the RAG knowledge base
7. User selects a solution and triggers proposal generation
8. Proposal Agent generates a full technical project document (Word, PDF, Notion)

2.3 Architectural Diagram (Text Representation)



3. Multi-Agent Architecture (LangGraph)

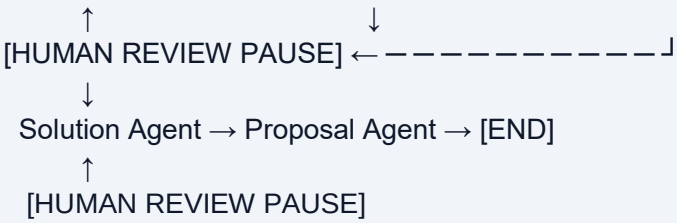
DivAi uses a LangGraph stateful graph as the orchestration backbone. Each agent is a specialised node in the graph. The Supervisor Agent manages routing between nodes, handles retries, and exposes human-in-the-loop pause points for user review.

3.1 Agent Graph Design

Following the LangGraph pattern from the AI Development Guide: nodes are Python functions that process shared state, edges define conditional routing, and MemorySaver provides session persistence across user interactions.

Graph Flow:

[START] → Supervisor → Ingestion Agent → Analysis Agent → Use Case Agent



3.2 Agent Definitions

Ingestion Agent

Role: Scrape target website — DOM structure, network requests, API endpoints, auth patterns, metadata

Inputs: Target URL, scrape depth (surface | deep | both), user session config

Outputs: Raw DOM tree (JSON), network request log (HAR format), extracted API endpoints list, page metadata

Tools / APIs: Playwright (headless browser), custom HTTP interceptor, JS execution context, robots.txt checker

Analysis Agent

Role: Process raw scrape data into structured intelligence — classify tech stack, data models, auth patterns, UI components

Inputs: Raw DOM JSON, HAR network log, API endpoints list

Outputs: Structured intelligence report: tech_stack[], api_endpoints[], data_schemas[], auth_method, ui_patterns[], integrations[]

Tools / APIs: Claude API (claude-sonnet-4-6), custom JSON schema validators, pattern matching classifiers

Use Case Agent

Role: Identify and rank business use cases from intelligence report using domain knowledge RAG

Inputs: Structured intelligence report, industry vertical (optional), client brief (optional)

Outputs: Ranked use case list: [{id, title, description, confidence_score, supporting_evidence, tags, effort_estimate}]

Tools / APIs: Claude API, ChromaDB RAG (use case knowledge base), LangChain retriever chain

Solution Agent

Role: Match selected use cases to solution blueprints from the RAG knowledge base and generate solution cards

Inputs: Selected use case IDs, client context, budget/timeline constraints
Outputs: Solution cards: [{id, title, pitch, architecture, stack, timeline, value_score, effort_score, risks}]
Tools / APIs: Claude API, ChromaDB RAG (solution blueprint library), LangChain retriever, pricing estimator tool

Proposal Agent

Role: Generate complete technical project proposal documents from selected solution card
Inputs: Selected solution card, client info, agency branding, output format preferences
Outputs: Formatted documents: .docx (python-docx), .pdf (WeasyPrint), Notion page (Notion API)
Tools / APIs: Claude API, python-docx, WeasyPrint, Notion API, document template system

Supervisor Agent

Role: Orchestrate the full pipeline — route between agents, manage human-in-the-loop pauses, handle errors and retries
Inputs: User actions, agent outputs, error signals
Outputs: Next node decision, state updates, user notifications, error recovery actions
Tools / APIs: LangGraph conditional edges, MemorySaver checkpointing, Redis pub/sub for real-time updates

4. Shared State Schema (LangGraph State)

All agents read from and write to a shared TypedDict state object. This is the single source of truth passed through every graph node. Using Pydantic models (as per Phase 7 of the AI Development Guide) ensures type safety and structured output.

```
# src/state/divai_state.py
from typing import TypedDict, Optional, List, Annotated
from pydantic import BaseModel

class APIEndpoint(BaseModel):
    method: str           # GET | POST | PUT | DELETE
    url: str              # endpoint path
    status: int           # HTTP status
    payload_schema: dict  # inferred request/response schema

class UseCase(BaseModel):
    id: str
    title: str
```

```
    description: str
    confidence_score: float   # 0.0 – 1.0
    supporting_evidence: List[str]
    tags: List[str]           # e.g. ['AI/ML', 'Automation']
    effort_estimate: str      # Low | Medium | High

class Solution(BaseModel):
    id: str
    title: str
    pitch: str
    stack: List[str]
    timeline: str
    value_score: str          # Low | High | Very High
    effort_score: str
    architecture_overview: str
    risks: List[str]

class DivAiState(TypedDict):
    # Input
    target_url: str
    scrape_depth: str         # surface | deep | both
    client_context: Optional[str]
    # Ingestion
    raw_dom: Optional[dict]
    network_log: Optional[list]
    # Analysis
    intelligence_report: Optional[dict]
    # Use Cases
    use_cases: Optional[List[UseCase]]
    selected_use_case_ids: Optional[List[str]]
    # Solutions
    solutions: Optional[List[Solution]]
    selected_solution_id: Optional[str]
    # Proposal
    proposal_paths: Optional[dict]  # {docx: path, pdf: path, notion: url}
    # Control
    current_stage: str
    error: Optional[str]
    messages: Annotated[list, 'conversation_history']
```

5. RAG System Design

RAG (Retrieval Augmented Generation) is used in two agents: the Use Case Agent and the Solution Agent. This grounds AI outputs in real domain knowledge accumulated by the agency, rather than relying purely on the LLM's general knowledge.

5.1 Knowledge Bases

Knowledge Base	Purpose	Contents	Used By
use_case_kb	Domain-specific use case patterns indexed by industry/tech signals	300+ use case templates with signal→opportunity mappings	Use Case Agent
solution_blueprint_kb	Agency solution blueprints with technical details and pricing	50+ solution cards: stack, timeline, architecture, risks	Solution Agent
proposal_template_kb	Document templates and writing patterns for proposals	15+ proposal sections with tone/structure examples	Proposal Agent

5.2 RAG Pipeline (per AI Development Guide Phase 5)

9. INDEXING (one-time): Load knowledge documents → chunk (RecursiveCharacterTextSplitter, 512 tokens, 50 overlap) → embed (Anthropic Embeddings) → store in ChromaDB
10. QUERYING (per request): Encode query as embedding → cosine similarity search in ChromaDB (top-k=5) → retrieve relevant chunks → inject into LLM prompt context
11. AGENTIC RAG: Use Case and Solution Agents are LangGraph subgraphs that decide when to query the KB vs. use LLM memory, enabling multi-step reasoning with document lookup

```
# src/05_rag/divai_use_case_rag.py  (pattern from Phase 5)
from langchain_anthropic import ChatAnthropic
from langchain_chroma import Chroma
from langchain.chains import RetrievalQA

vectorstore = Chroma(
    collection_name='use_case_kb',
    embedding_function=AnthropicEmbeddings(),
    persist_directory='./data/chroma'
)
retriever = vectorstore.as_retriever(search_kwargs={'k': 5})
chain = RetrievalQA.from_chain_type(
    llm=ChatAnthropic(model='claude-sonnet-4-6'),
    retriever=retriever,
    return_source_documents=True
)
```

6. Backend API Design (FastAPI)

The backend is a FastAPI application that exposes the LangGraph pipeline as a REST API with streaming support. It follows the production patterns from Phase 7 of the AI Development Guide.

6.1 API Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/scan	Initiate a website scrape and analysis job. Returns job_id.	Bearer JWT
GET	/api/v1/scan/{job_id}	Poll scan status and retrieve intelligence report when complete.	Bearer JWT
GET	/api/v1/scan/{job_id}/stream	Server-Sent Events stream for real-time agent progress updates.	Bearer JWT
POST	/api/v1/usecases/{job_id}	Trigger use case generation from a completed analysis.	Bearer JWT
POST	/api/v1/solutions	Generate solution cards for selected use case IDs.	Bearer JWT
POST	/api/v1/proposal	Generate proposal document from selected solution.	Bearer JWT
GET	/api/v1/proposal/{proposal_id}/download	Download generated .docx or .pdf file.	Bearer JWT
POST	/api/v1/proposal/{proposal_id}/notion	Push proposal to Notion workspace.	Bearer JWT
GET	/api/v1/health	Health check endpoint for load balancer.	None

6.2 Streaming Pattern

Agent progress is streamed to the client using Server-Sent Events (SSE) via FastAPI's StreamingResponse. This provides real-time feedback as each agent node executes.

```
# src/api/routes/scan.py
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

@app.get('/api/v1/scan/{job_id}/stream')
async def stream_scan_progress(job_id: str):
    async def event_generator():
        async for event in divai_graph.astream(config):
            yield f'data: {json.dumps(event)}\n\n'
    return StreamingResponse(
        event_generator(), media_type='text/event-stream'
    )
```

7. Frontend Architecture (Next.js / TypeScript)

The frontend is a Next.js 14 application using the App Router, TypeScript throughout, and the Anthropic TypeScript SDK for any client-side AI features. It consumes the FastAPI backend via REST/SSE.

7.1 Page Structure

Route	Component	Purpose
/	LandingPage	URL input, product overview, scan initiation
/scan/[jobId]	ScanPage	Live scrape progress via SSE stream
/analyze/[jobId]	AnalyzePage	DOM + network intelligence report display
/usecases/[jobId]	UseCasesPage	Use case cards with confidence scores, multi-select
/solutions/[jobId]	SolutionsPage	Solution cards, effort/value matrix
/proposal/[proposalId]	ProposalPage	Proposal preview, export options
/dashboard	DashboardPage	History of all scans and proposals
/admin	AdminPage	Knowledge base management, template editor

7.2 Key Frontend Patterns

- TypeScript SDK: Use @anthropic-ai/sdk for any client-side streaming or in-browser AI features
- Zod schemas: Mirror backend Pydantic models for end-to-end type safety
- SWR / React Query: Data fetching with optimistic updates and cache invalidation

- SSE hook: Custom useAgentStream() hook that subscribes to /scan/{id}/stream and updates component state
- Role-based views: Internal users see confidence scores and tech details; client-facing view shows business-level summaries

8. Data Layer

8.1 Database Schema (PostgreSQL)

Table	Key Columns	Purpose
users	id, email, role (internal|client), org_id, created_at	Auth and access control
organisations	id, name, notion_token, branding_config	Multi-tenant client orgs
scan_jobs	id, url, status, scrape_depth, created_by, created_at, completed_at	Track scrape jobs
intelligence_reports	id, job_id, tech_stack, api_endpoints (JSONB), data_schemas (JSONB)	Structured analysis output
use_cases	id, job_id, title, description, confidence, tags (JSONB), selected	Generated use cases
solutions	id, job_id, use_case_ids (JSONB), title, stack (JSONB), timeline, value_score	Generated solutions
proposals	id, solution_id, docx_path, pdf_path, notion_url, status, created_at	Proposal documents

8.2 Vector Store (ChromaDB)

Engine	ChromaDB (local dev), Pinecone (production)
Collections	use_case_kb, solution_blueprint_kb, proposal_template_kb
Embedding Model	Anthropic Embeddings (text-embedding-3)
Chunk Size	512 tokens with 50-token overlap
Retrieval	Cosine similarity, top-k=5 per query

8.3 Cache Layer (Redis)

- Session state: LangGraph MemorySaver uses Redis for persistent graph checkpointing across API requests
- Job status: Scan job status cached for SSE polling performance
- Rate limiting: Per-user scrape rate limits enforced at Redis layer
- Pub/Sub: Agent progress events published to Redis channels, consumed by SSE endpoint

9. Website Ingestion Deep Dive

The Ingestion Agent is the most technically complex component. It uses Playwright for headless browser control to capture both the rendered DOM and all network traffic, mirroring what the Inspect Element and Network DevTools tabs expose.

9.1 What Gets Captured

Signal Type	What Is Captured	Insight Generated
DOM Structure	HTML tag hierarchy, React/Vue component markers, CSS class patterns, form elements	UI framework, page types, feature surface
Network Requests	All XHR/fetch calls: method, URL, headers, payload schemas, response structure	API endpoints, data models, auth mechanism
JavaScript Globals	window.__NEXT_DATA__, Redux store shape, global config objects	Framework, feature flags, data structure
Auth Patterns	Auth headers (Bearer, Cookie), token refresh patterns, redirect flows	Auth method, session architecture
Third-Party Scripts	Analytics, payment gateways, CRM embeds, chat widgets (by script src)	Existing integrations, tech stack
Meta & SEO	og:tags, robots.txt, sitemap.xml, structured data (JSON-LD)	Business vertical, content architecture

9.2 Playwright Scrape Pattern

```
# src/agents/ingestion_agent.py
from playwright.async_api import async_playwright

async def scrape_website(url: str, depth: str) -> dict:
```

```
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True)
        context = await browser.new_context()
        # Intercept ALL network requests
        requests_log = []
        context.on('request', lambda r: requests_log.append({
            'method': r.method, 'url': r.url,
            'headers': dict(r.headers)
        }))
        page = await context.new_page()
        await page.goto(url, wait_until='networkidle')
        # Capture DOM
        dom = await page.evaluate('() => document.documentElement.outerHTML')
        # Capture JS globals for deep scrape
        if depth in ('deep', 'both'):
            globals = await page.evaluate('() =>
JSON.stringify(window.__NEXT_DATA__ || {})')
        await browser.close()
        return {'dom': dom, 'network': requests_log, 'js_globals': globals}
```

10. Proposal Generation System

10.1 Output Formats

Format	Library	Template Approach	Trigger
Word (.docx)	python-docx	Jinja2 template → python-docx renderer with agency branding	Default — always generated
PDF (.pdf)	WeasyPrint	HTML/CSS template rendered to PDF via headless Chromium	Generated from DOCX
Notion Page	Notion API (SDK)	Block-by-block page creation using Notion block types	User triggers export to Notion
Interactive Web	React component	Rendered in-app proposal viewer with edit capability	Always available in-app

10.2 Proposal Document Sections

Every generated proposal contains the following standardised sections. The Proposal Agent uses the proposal_template_kb RAG knowledge base to ensure tone, depth, and technical accuracy match agency standards.

- Executive Summary — Business problem, proposed solution, key outcomes

- Problem Statement — Evidence from website analysis supporting the need
- Proposed Architecture — System design overview, data flow diagram (text)
- Technical Stack — Chosen technologies with rationale
- Phase Breakdown — Sprints, milestones, deliverables per phase
- API Specifications — Key endpoints and integration points
- Security & Compliance — Auth strategy, data handling, GDPR notes
- Testing Strategy — Unit, integration, E2E test coverage plan
- Deployment & DevOps — Infrastructure, CI/CD pipeline, environments
- Effort Estimate — Hours breakdown by role (dev, design, QA, PM)
- Investment Summary — Cost range with assumptions
- Assumptions & Risks — Open questions, dependencies, mitigation
- Next Steps — Action items, sign-off requirements

11. Infrastructure & Deployment

11.1 Environment Overview

Environment	Purpose	Infrastructure
Development	Local dev with hot reload	Docker Compose (FastAPI + Postgres + Redis + ChromaDB)
Staging	QA and client demos	Railway / Render — single-region deployment
Production	Live platform	AWS ECS (Fargate) + RDS (Postgres) + ElastiCache (Redis) + S3

11.2 Docker Compose (Development)

```
# docker-compose.yml
services:
  api:          # FastAPI backend
    build: ./backend
    ports: ['8000:8000']
    env_file: .env
    depends_on: [postgres, redis, chromadb]
  frontend:     # Next.js
    build: ./frontend
    ports: ['3000:3000']
  postgres:     # PostgreSQL
    image: postgres:16
    volumes: ['pgdata:/var/lib/postgresql/data']
```

```
  redis:         # Cache + LangGraph memory
    image: redis:7
  chromadb:      # Vector store
    image: chromadb/chroma
    volumes: ['chromadata:/chroma/data']
```

11.3 CI/CD Pipeline

12. Push to feature branch → GitHub Actions triggers lint, type-check, unit tests
13. PR to main → Integration tests run, Playwright E2E tests against staging
14. Merge to main → Auto-deploy to staging environment
15. Tagged release → Manual approval → Deploy to production

12. Security Design

Concern	Approach
Authentication	JWT Bearer tokens (access + refresh). OAuth2 via NextAuth.js on frontend.
Authorisation	Role-based: internal (full access) vs. client (read-only proposals, no raw data)
API Keys	Anthropic API key stored in AWS Secrets Manager, injected at runtime via env
Scrape Ethics	robots.txt parsed and honoured. Rate limiting on scrape requests. No personal data stored.
Data Isolation	PostgreSQL row-level security by org_id. Separate ChromaDB collections per org.
Document Security	Generated .docx/.pdf stored in private S3 bucket with pre-signed time-limited URLs
Input Validation	All user inputs validated with Pydantic (Python) and Zod (TypeScript) before processing

13. Phased Delivery Roadmap

The delivery roadmap is aligned with the learning phases in the AI Development Guide, ensuring each phase builds genuine understanding before advancing to production complexity.

Phase 1: Foundations & Ingestion | Weeks 1–2

- Set up Python venv, install requirements, configure Anthropic API key
- Build Ingestion Agent: Playwright scraper, DOM capture, network request logger
- Direct API call (Phase 1): Feed DOM to Claude, get raw analysis back
- Unit tests for scraper, mock website test fixtures
- Deliverable: CLI tool — input URL, output raw intelligence JSON

Phase 2: Analysis & Tool Use | Weeks 3–4

- Build Analysis Agent using Tool Use pattern (Phase 2 of guide)
- Define tools: classify_tech_stack, extract_api_endpoints, detect_auth_pattern
- Implement agent loop: LLM decides which analysis tools to call
- Structured output with Pydantic (intelligence_report schema)
- Deliverable: Analysis Agent that returns typed IntelligenceReport from any URL

Phase 3: Use Case & Solution Agents (LangChain) | Weeks 5–6

- Set up ChromaDB, seed use_case_kb and solution_blueprint_kb with initial data
- Build RAG pipeline (Phase 5): embed knowledge docs, create retriever chains
- Use Case Agent: LangChain RetrievalQA chain → ranked use case list
- Solution Agent: RAG retrieval + LangChain chain → solution cards
- LangChain chains (Phase 3): prompt templates, output parsers, LCEL pipes
- Deliverable: End-to-end CLI: URL → use cases → solutions

Phase 4: LangGraph Orchestration | Weeks 7–8

- Implement full LangGraph stateful graph (Phase 4 of guide)
- Wire all agents as graph nodes with DivAiState shared state
- Add conditional edges, human-in-the-loop interrupts for review stages
- Implement MemorySaver with Redis for session persistence
- Supervisor Agent: routing logic, error handling, retry policies
- Deliverable: Full orchestrated pipeline running end-to-end in Python

Phase 5: Proposal Generation & Exports | Weeks 9–10

- Proposal Agent: python-docx document generation with agency branding
- PDF export via WeasyPrint, S3 storage, pre-signed URL download
- Notion API integration — push proposal as Notion page
- Proposal template RAG: ensure tone and structure match agency standards
- Deliverable: Full proposal generation in .docx, .pdf, and Notion formats

Phase 6: FastAPI Backend & TypeScript Frontend | Weeks 11–14

- FastAPI REST API wrapping LangGraph pipeline (Phase 7 production patterns)

- PostgreSQL schema, Alembic migrations, SQLAlchemy ORM
- SSE streaming endpoint for real-time agent progress
- Next.js 14 frontend (Phase 6 TypeScript): all pages, SSE hook, Zod schemas
- Auth: JWT + NextAuth.js, role-based views (internal vs. client-facing)
- Deliverable: Full-stack web application, deployable via Docker Compose

Phase 7: Production, Monitoring & Launch | Weeks 15–16

- LangSmith integration (Phase 7): trace and evaluate all LLM calls
- AWS deployment: ECS Fargate, RDS, ElastiCache, S3, Secrets Manager
- CI/CD pipeline: GitHub Actions → staging → production
- Performance testing: load test scrape jobs, optimise Playwright pool
- Admin panel: knowledge base management, template editor
- Deliverable: Production-ready DivAi platform, live and monitored

14. Full Technology Stack

Layer	Technology	Version	Purpose
AI / LLM	Claude (claude-sonnet-4-6)	Latest	Core reasoning, analysis, generation
AI / LLM	Anthropic Python SDK	0.40+	Direct API calls (Phase 1 & 2)
Orchestration	LangGraph	0.2+	Stateful multi-agent graph
Chains	LangChain	0.3+	Prompt templates, LCEL chains, tool use
Vector Store	ChromaDB (dev)	0.5+	Local RAG knowledge base
Vector Store	Pinecone (prod)	Latest	Cloud-scale vector search
Backend	FastAPI	0.115+	REST API, SSE streaming
Database	PostgreSQL	16	Relational data store
Cache / Memory	Redis	7	LangGraph checkpointing, pub/sub
Scraping	Playwright	1.45+	Headless browser, network interception
Documents	python-docx	1.1+	DOCX proposal generation
PDF	WeasyPrint	62+	HTML-to-PDF rendering
Integrations	Notion SDK	2.2+	Notion page creation
Frontend	Next.js 14	14+	React SSR framework

Frontend	TypeScript	5+	Type-safe frontend code
Frontend	Anthropic TS SDK	0.27+	Client-side AI features
Validation	Pydantic	2+	Python schema validation
Validation	Zod	3+	TypeScript schema validation
Monitoring	LangSmith	Latest	LLM tracing and evaluation
Infrastructure	Docker / Docker Compose	Latest	Local dev orchestration
Infrastructure	AWS ECS + RDS + S3	Latest	Production cloud infrastructure
CI/CD	GitHub Actions	Latest	Automated test and deploy pipeline

15. Immediate Next Steps

Action Items to Begin Development:

1. Clone the DivAi repo, run: python -m venv venv && pip install -r requirements.txt
2. Copy .env.example to .env, add ANTHROPIC_API_KEY from console.anthropic.com
3. Complete Phase 1 basics: run src/01_basics/01_first_api_call.py successfully
4. Set up Playwright: pip install playwright && playwright install chromium
5. Build the first Ingestion Agent (Week 1 task) — scrape a real website
6. Seed ChromaDB with 10+ use case templates to enable RAG from Week 5
7. Stand up Docker Compose stack with Postgres + Redis + ChromaDB

This document is a living specification. It should be updated as architectural decisions evolve throughout the build. Version history should be maintained in the document header.

Document prepared by DivAi Engineering | Based on the AI Development Guide (Python · LangChain · LangGraph · RAG · TypeScript)